

Detekcija objekata od interesa u prometu



Sadržaj

- Viola-Jones (V-J) algoritam za detekciju objekata
 - Haarove značajke
 - Integralna slika
 - Adaboost algoritam
 - Kaskada klasifikatora
- Implementacija algoritma u OpenCV i primjeri
 - Detekcija lica i očiju
 - Detekcija pješaka
 - Detekcija automobila

Podsjetimo se detekcije objekata...

- **Metode i tehnike detekcije objekata obrađene do sada**
 - Pronalazak uglova, SIFT, SURF, FAST, BRIEF, ORB, HoG,...
 - Možemo izdvojiti određene značajke iz slike
 - Izdvojene značajke možemo koristiti da detektiramo i/ili klasificiramo objekte

Viola-Jones algoritam

Viola-Jones algoritam

- Osnovne informacije
 - Objavljen u znanstvenim radovima [1] i [2] → preko 20000 citata
 - **Metoda za brzo pronalaženje objekata u slici korištenjem kaskade „jakih” Haarovih klasifikatora**
 - **Primarno napravljen za otkrivanje ljudskih lica na slici**
 - Koristi algoritme strojnog učenja → može se koristiti za pronalaženje bilo koje vrste objekata
 - **Pješak, automobil, semafor, prometni znak, pješački prijelaz, ...**
 - Potrebno je istrenirati klasifikatore prije toga
 - **Robustan → visoka brzina detekcije i mali broj pogrešnih detekcija**
 - **Radi u stvarnom vremenu**
 - Koristi Haarove značajke

Viola-Jones algoritam

- Na čemu je zasnovan?
 - Učenje (edukacija) klasifikatora strojnim učenjem
 - Pozitivne i negativne slike
 - **Pozitivne slike (sadrže objekt od interesa)**
 - **Negativne slike (ne sadrže objekt od interesa)**
 - Aplikaciji se daje informacija o tome koja slika sadrži objekt za koji se klasifikator trenira, a koja ne sadrži (tzv. labele)

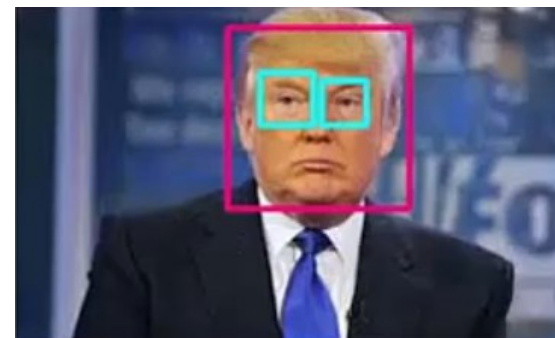
Viola-Jones algoritam

- Na kojim je koracima zasnovan Viola-Jones algoritam ?
 - Haarove značajke
 - Integralna slika
 - AdaBoost algoritam
 - Kaskadni klasifikator
- Dvije faze algoritma
 - **Treniranje**
 - Testiranje

Haarove značajke i kaskadni klasifikatori

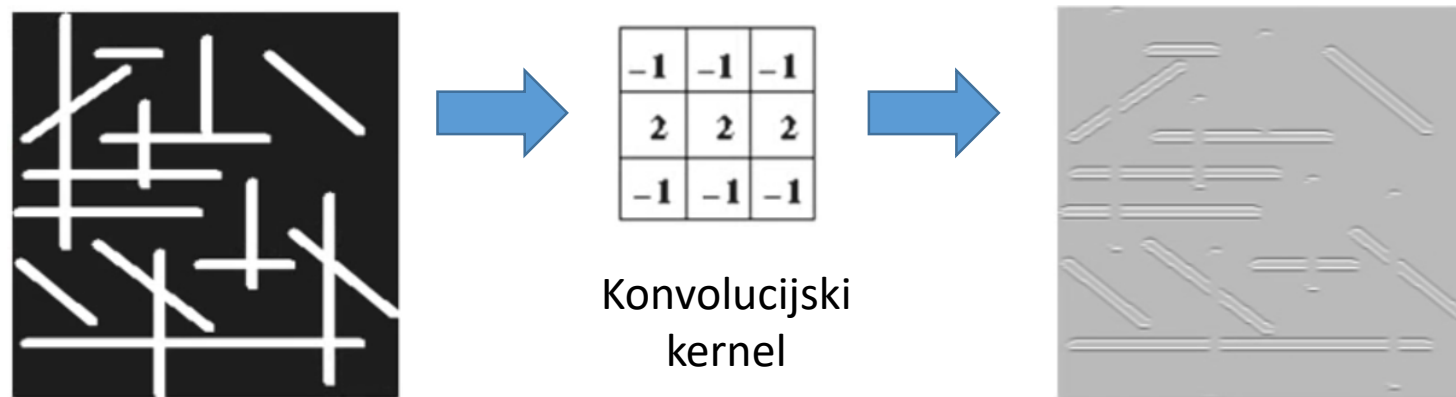
Što su Haarove značajke i Haarovi kaskadni klasifikatori (engl. *Haar cascade classifiers* - HCC) i kako rade ?

- Metoda za detekciju objekata koja ima drugačiji način spremanja (pohrane) značajki – te se značajke zovu Haarove značajke
- Koriste se Haarove značajke u nizu klasifikatora (kaskadi) kako bi identificirali objekt u slici (je li objekt prisutan u slici ili nije)
- Klasifikatori su istrenirani da identificiraju jedan tip objekta
- Može se koristiti nekoliko klasifikatora paralelno (npr. detekcija očiju i lica zajedno)



Haarove značajke

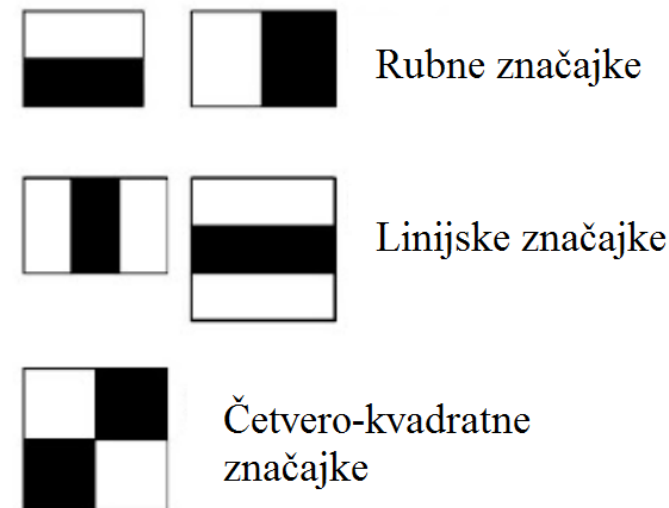
- Detekcija objekata → umjesto izravnog rada s pikselima (usporedba s pikselima iz trening skupa), uveden je rad sa značajkama koje imaju pravokutni oblik → računalno učinkovitiji
- Haarove značajke → slične konvolucijskim kernelima koje smo spominjali npr. kod detekcije rubova, linija,..., a koji se koriste za detekciju određene značajke u danoj slici



- Izlazna slika ima visoki intenzitet na mjestima piksela gdje uzorak elemenata konvolucijskog kernela savršeno odgovara ulaznoj slici
- Obratite pažnju da vertikalne linije nisu detektirane s ovim kernelom

Haarove značajke

- Osnovni tipovi Haarovih značajki u Viola-Jones algoritmu
- Gledano kao konvolucijski kernel $\rightarrow$ crno područje ima vrijednosti 1, bijelo -1
- Sve ostale značajke dobivaju se translacijom i skaliranjem osnovnih značajki (ili rotacijom)
- Primjena svake značajka rezultira jedinstvenom vrijednošću koja je izračunata oduzimanjem sume intenziteta piksela ispod bijelog pravokutnika od sume intenziteta piksela ispod crnog pravokutnika



Haarove značajke

- Primjena na danu sliku...

Tip 1



Tip 2



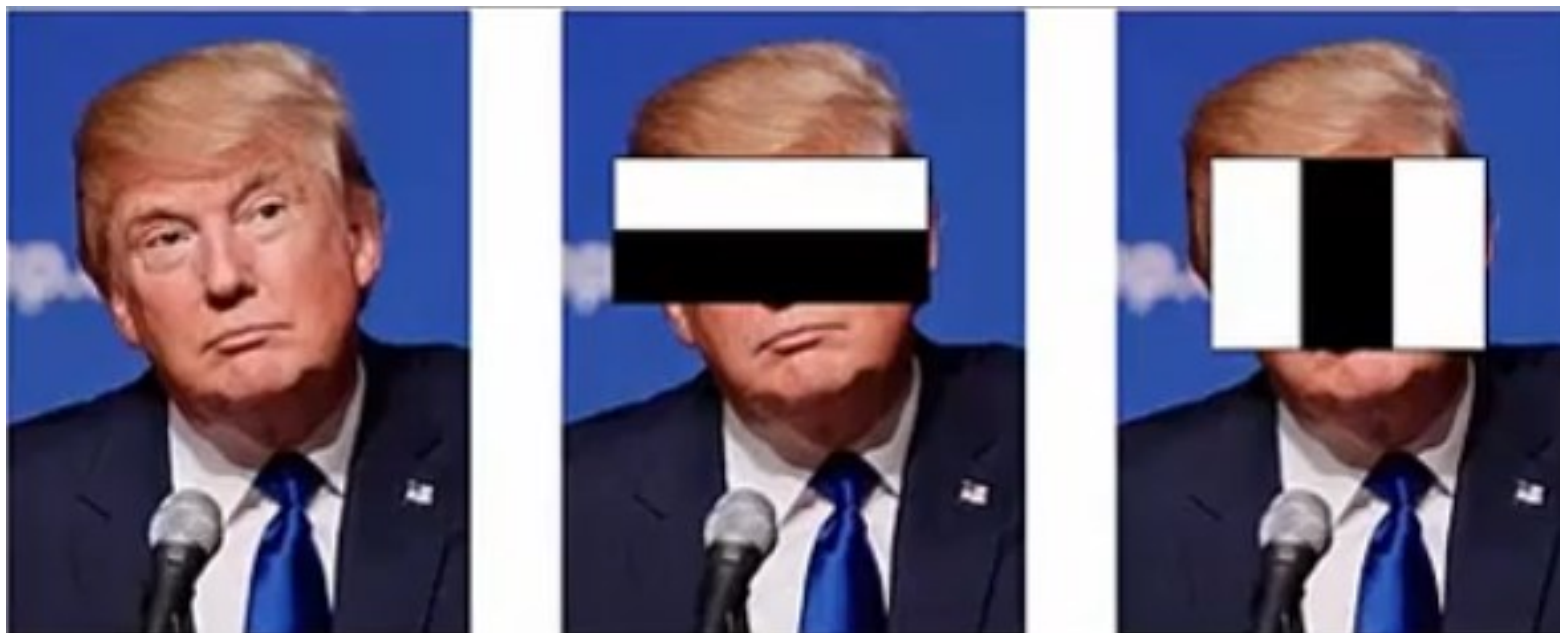
Tip 3



Tip 4



Tip 5



Haarove značajke

- Primjena na danu sliku...

Tip 1



Tip 2



Tip 3



Tip 4



Tip 5



Haarove značajke

Tip 1



Tip 2



Tip 3



Tip 4



Tip 5



- Npr. značajka Tipa 3 (zadnja slika) → namijenjena da detektira nos (most nosa)
- Područje nosa je svjetlije od ruba nosa, koji je tamniji → ova će značajka izdvojiti područje nosa
- Kada se značajka Tipa 3 primjeni na cijelu sliku, dat će velike vrijednosti samo na lokacijama onih piksela koji potencijalno pripadaju nosu
- Usporedba vrijednosti koju daje primjena značajke Tipa 3 na određenoj lokaciji sa zadanim pragom P_3 → rezultate je binarna odluka o postojanju nosa (postoji/ne postoji nos na toj lokaciji) → binarni klasifikator

Haarove značajke

Tip 1



Tip 2



Tip 3



Tip 4



Tip 5



- Slično → značajka Tipa 2 za detekciju očiju → svijetla područja ispod očiju i tamna područja samih očiju
- Kada se značajka Tipa 2 primjeni na cijelu sliku, dat će velike vrijednosti samo na lokacijama onih piksela koji potencijalno pripadaju očima
- Usporedba vrijednosti koju daje primjena značajke Tipa 2 na određenoj lokaciji sa zadanim pragom P_2 – binarna odluka o postojanju očiju → binarni klasifikator

Haarove značajke

Tip 1



Tip 2



Tip 3



Tip 4



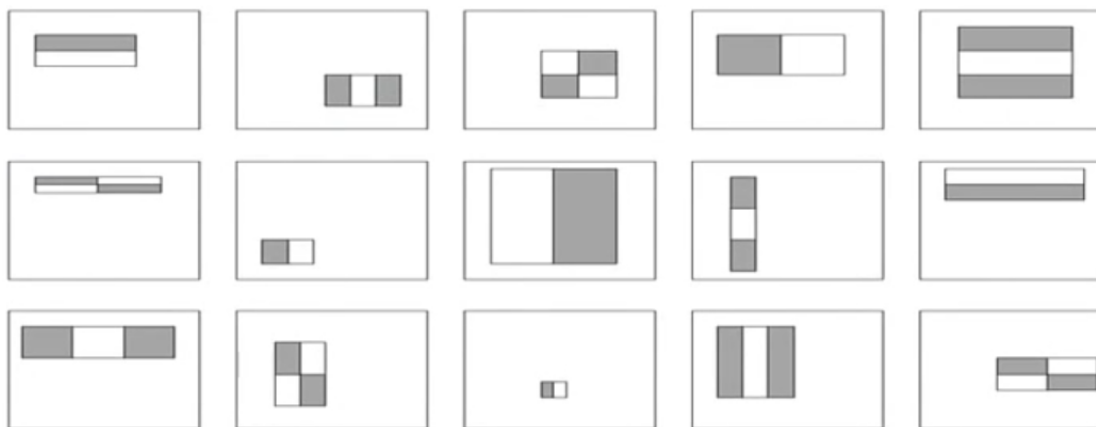
Tip 5



- Svi ti tipovi značajki predstavljaju neku od karakteristika lica
- Činjenica → primjena pojedinog Haarovog klasifikatora (značajke) samostalno ne bi dovela do uspješne detekcije objekata
- Zaključak → kombinacijom više klasifikatora (značajki) moglo bi se puno uspješnije detektirati objekte
 - Kako to napraviti ???

Haarove značajke

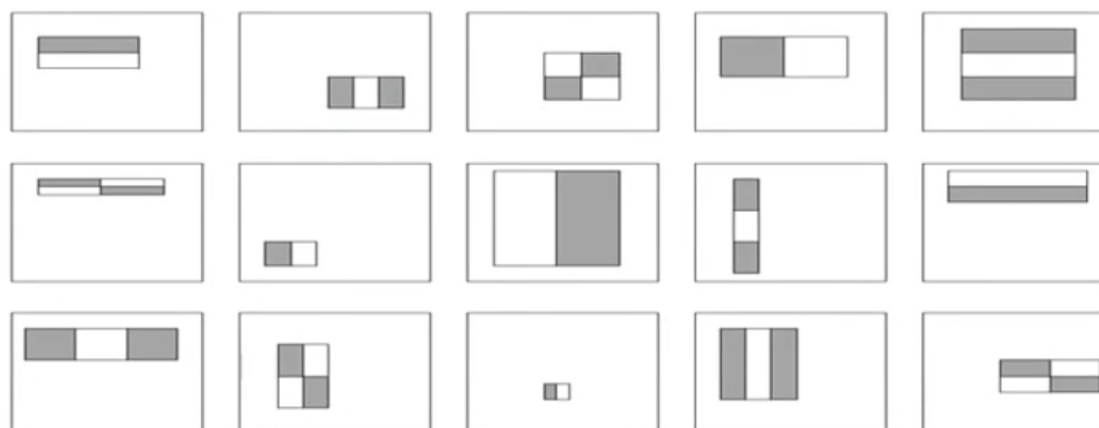
- Viola Jones algoritam koristi prozor 24x24 piksela kao osnovni prozor kako bi počeo pretraživanje tih značajki u danoj slici



- Da bi bio u mogućnosti detektirati objekte različite veličine u slici, potrebno je prilikom pretrage značajki čini jednu od sljedeće 2 stvari:
 - Mijenjati veličinu značajki, s npr. 1x2 na 2x2, 4x2, 8x2,...
 - Mijenjati rezoluciju slike (smanjivati sliku)

Haarove značajke

- Viola Jones algoritam koristi prozor 24x24 piksela kao osnovni prozor kako bi počeo pretraživanje tih značajki u danoj slici



- Ako se razmotre svi mogući parametri Haarovih značajki poput skale, tipa i lokacije, dolazi se do računanja **160 000+** značajki u tom prozoru 24x24 piksela

Haarove značajke

- Ovdje dolazimo do dva glavna problema problema...

(1) Moramo računati mnoštvo suma piksela pod „crnim regijama” i pod „bijelim regijama” za sve te pravokutne Haarove značajke (a ima ih **160000+ za svaki pod-prozor slike 24x24 piksela)**

- Nije računalno učinkovito za stvarno-vremensku upotrebu
- Ideja – smanjiti broj proračuna tj. ne sumirati sve piksele ispod „bijelih” i „crnih” područja svaki puta kada računamo neku značajku
- Rješenje → tehnika/koncept koja se zove **Integralna slika (*Integral Image*)**

Haarove značajke

- Ovdje dolazimo do dva glavna problema problema...

(2) Moramo procijeniti važnost velikog broja proračunatih značajki → **160 000+** značajki za svaki pojedini podprozor slike veličine 24x24 piksela

- Praktički neizvedivo u stvarnom vremenu
- Ideja → eliminirati mnoštvo značajki koje su irelevantne tj. koje nisu korisne i odabrati samo značajke koje su nam korisne → to radi AdaBoost algoritam → daje na kraju nekoliko tisuća značajki umjesto 160 000+

P i t a n j a ?

Integralna slika

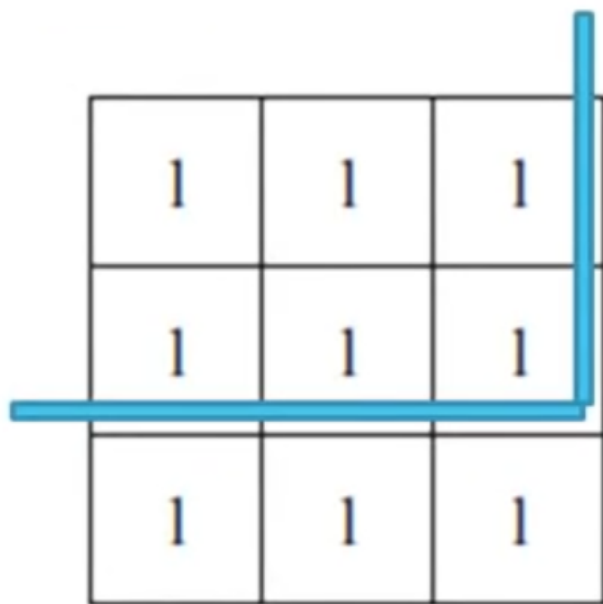
- Proračnuava se na samom početku postupka, prije proračuna **160000+** Haarovih značajki, kako bi ubrzali njihov proračun
- **Ideja → pronaći sumu svih piksela unutar nekog pravokutnika uz samo 4 vrijednosti piksela u integralnoj slici**
- Kada imamo integralnu sliku i želimo izračunati npr. sumu svih piksela „prekrivenih” crnim uzorkom na slici ispod, ne moramo računati sumu svih piksela pod crnim dijelom



- Dovoljne su nam samo 4 rubne vrijednosti elemenata integralne slike da to izračunamo
- Kako se dobije integralna slika ???

Integralna slika

- U integralnoj slici vrijednost na lokaciji piksela (x,y) jednaka je sumi vrijednosti svih piksela iznad i lijevo od lokacije (x,y) , uključujući i piksel na lokaciji (x,y)
- Primjer...



1	1	1
1	1	1
1	1	1

Ulazna slika

Suma elemenata
slike iznad i lijevo
(uključujući i dani
element slike)

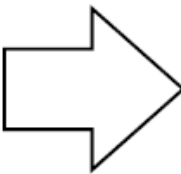
1	2	3
2	4	6
3	6	9

Integralna slika

Integralna slika

- U integralnoj slici vrijednost na lokaciji piksela (x,y) jednaka je sumi vrijednosti svih piksela iznad i lijevo od lokacije (x,y) , uključujući i piksel na lokaciji (x,y)
- Primjer...

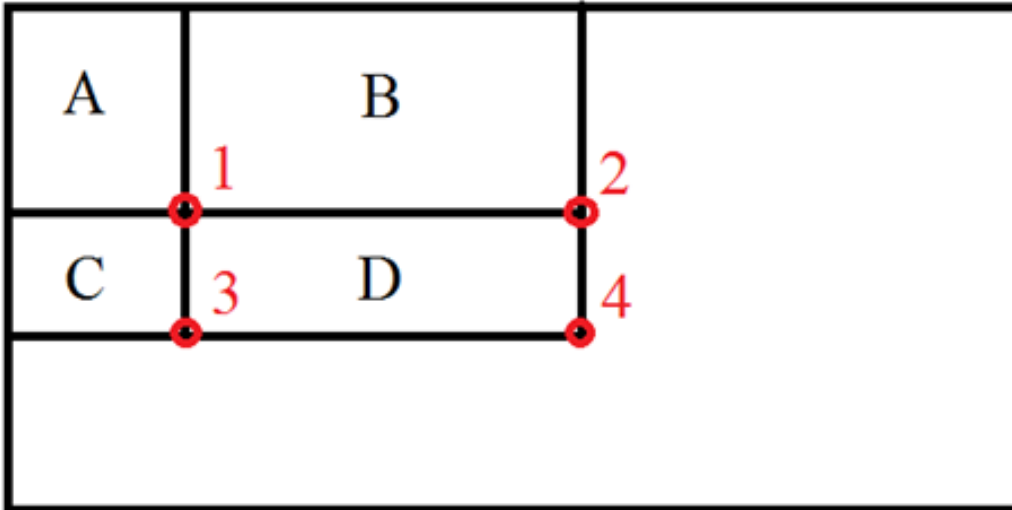
0	1	1	1
1	2	2	3
1	2	1	1
1	3	1	0



0	1	2	3
1	4	7	11
2	7	11	16
3	11	16	21

Integralna slika

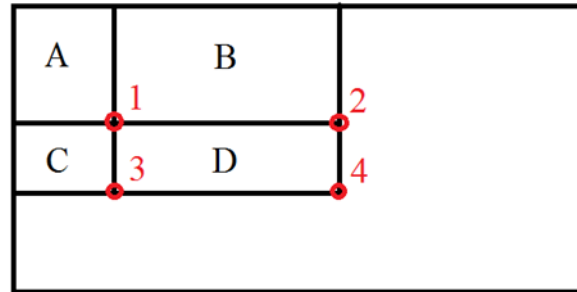
- U redu, ali još uvijek nije jasno zašto se ovo radi, koja je prednost ?
- Integralna slika omogućava izračun sume svih piksela unutar bilo kojeg pravokutnog područja koristeći samo 4 vrijednosti u uglovima pravokutnika
- To čini ovu tehniku super-brzom



Suma svih elemenata slike u

$$\begin{aligned} D &= 1 + 4 - (2 + 3) \\ &= A + (A + B + C + D) - ((A + B) + (A + C)) \\ &= D \end{aligned}$$

Integralna slika



Suma svih elemenata slike u

$$D = 1 + 4 - (2 + 3)$$
$$= A + (A + B + C + D) - ((A + B) + (A + C))$$
$$= D$$

- Za izračun Haarove značajke s dva pravokutnika, koliko je puta potrebno pristupiti integralnoj slici?

- Odgovor: 6



- Za izračun Haarove značajke s tri pravokutnika, koliko je puta potrebno pristupiti integralnoj slici?

- Odgovor: 8



- Za izračun Haarove značajke s četiri pravokutnika, koliko je puta potrebno pristupiti integralnoj slici?

- Odgovor: 9

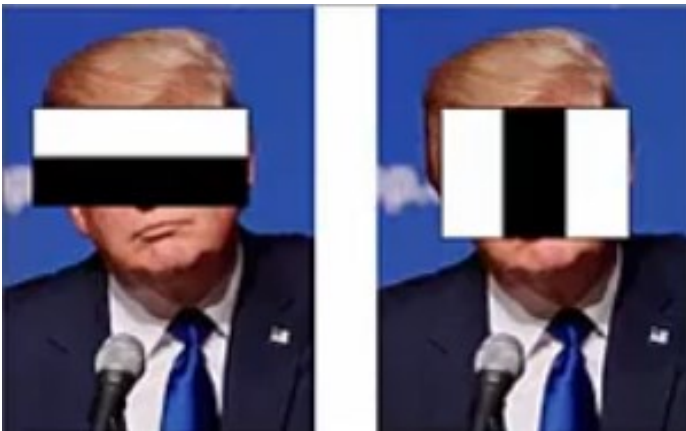


**Integralna slika znatno
ubrzava proračun značajki**

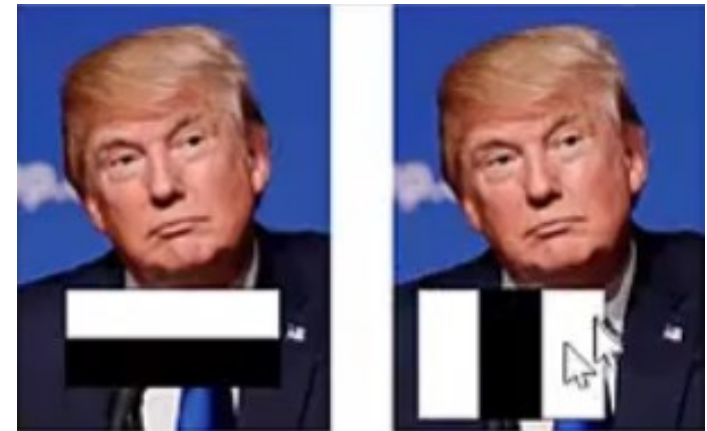
P i t a n j a ?

AdaBoost algoritam

- U redu, sad smo izračunali vrlo brzo i efikasno **160000+** značajki za svaki podprozor 24x24 elementa slike
- No jesu li nam sve jednako bitne? Jesu li sve relevantne/potrebne? Trebaju li nam sve one u svakom podprozoru da bi odlučili potosji li lice u njemu ili ne?
- Očekivano → samo neke od tih značajki bit će korise (relevantne) za identifikaciju lica → njih Adaboost algoritam pokušava identificirati



Relevantna



Irelevantna

AdaBoost algoritam



Sve značajke



Relevantna



Irelevantna

AdaBoost algoritam

- Adaboost je algoritam strojnog učenja
- Pomaže u pronalaženju samo najboljih/najvažnijih značajki u skupu od **160000+** značajki za svaki podprozor slike 24x24 piksela
- Gradi jadan jaki klasifikator kao linearnu kombinaciju slabih klasifikatora
- Npr. na temelju kombinacije binarnih odluka pojedinih klasifikatora o postojanju pojedinih značajki lica u podprozoru 24x24 piksela, donosi konačnu odluku postoji li u tom podprozoru lice ili ne postoji
 - Nos DA/NE
 - Oči DA/NE
 - Usta DA/NE
 -konačna odluka → Lice DA/NE

Ideja Adaboost algoritma – stvaranje JAKOG klasifikatora pomoću više SLABIH klasifikatora (koji su povezani s pojedinim značajkama)

AdaBoost algoritam

- Ideja Adaboost algoritma – stvaranje JAKOG klasifikatora pomoću više SLABIH klasifikatora (koji su povezani s pojedinim značajkama)
- Što je SLABI klasifikator ?
 - Svaka značajka se smatra slabim klasifikatorom h_n i uključuje u skup odabranih ako radi bolje od slučajnog odabira tj. ako daje ispravan izlazni rezultat za više od 50% uzoraka
 - Izlaz slabog klasifikatora je binaran (ovisno što značajka pokušava detektirati)
 - Postoji most nosa ili ne (izlaz je 0 ili 1)
 - Postoje oči ili ne (izlaz je 0 ili 1)...
- JAKI klasifikator → dobiva se linearnom kombinacijom slabih klasifikatora

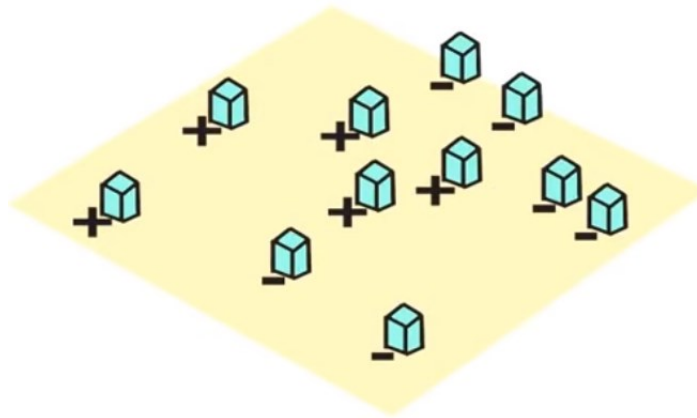
$$H(x) = \sum_{n=1}^N a_n h_n(x)$$

AdaBoost algoritam

- Kako se gradi **JAKI** klasifikator (npr. za detekciju lica u slici)?
- Inicijalno → potrebna ljudska pomoć, tj. učenje računala da li na nekoj slici postoji lice ili ne postoji
- Početni skup trening slika (trening skup), tj uređenih parova (x_i, y_i)
 - x_i – i-ta slika
 - y_i – 0 ako je i-ta slika negativna, 1 ako je i-ta slika pozitivna (npr. sadrži lice)

AdaBoost algoritam

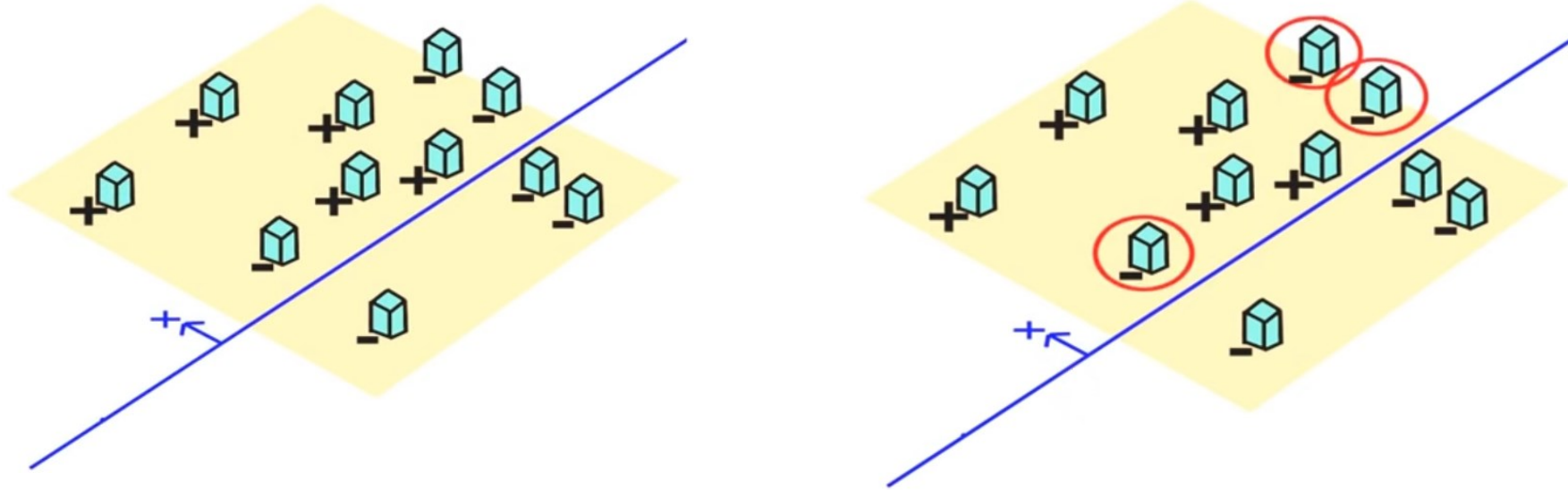
- Primjer - neka imamo 11 slika (5 slika sadrže lice, 6 ne sadrže)



- Adaboost za početak pridijeli svim slikama (trening uzorcima) jednake težine (na slici prikazane visinom kvadra)
- Na kraju → želi kombinirati različite značajke kako bi identificirao slike s licem

AdaBoost algoritam

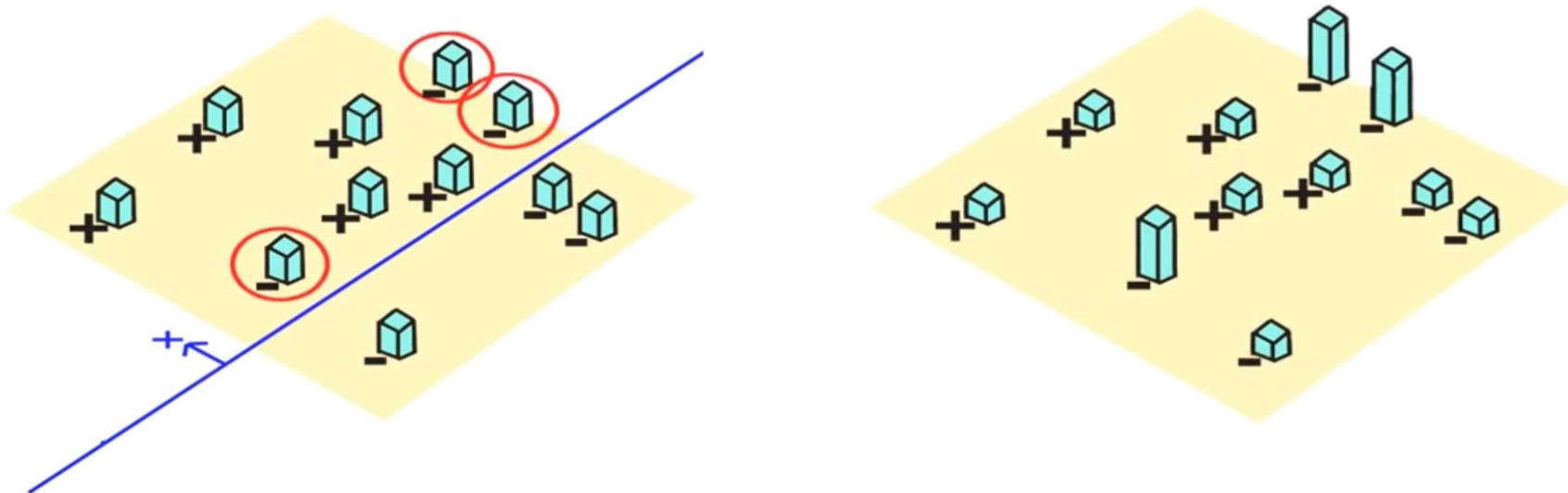
- Odabiremo prvi slabi klasifikator h_1 kojim su slike klasificirane



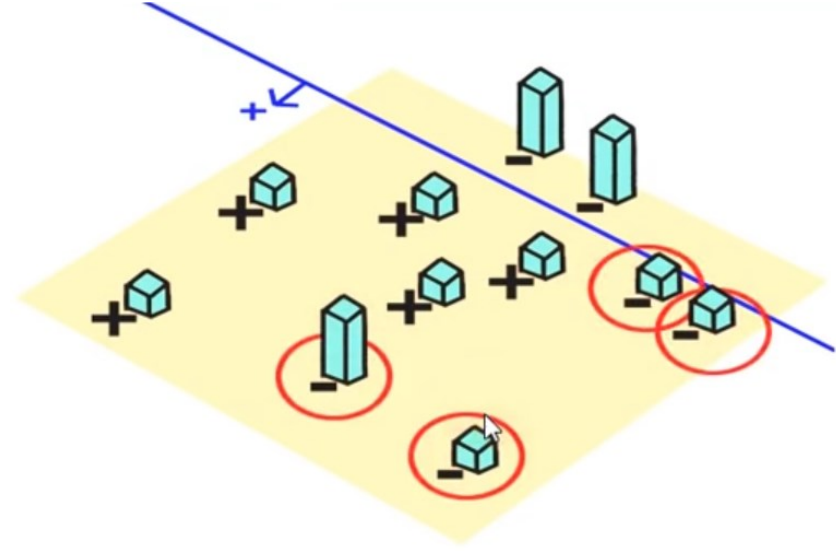
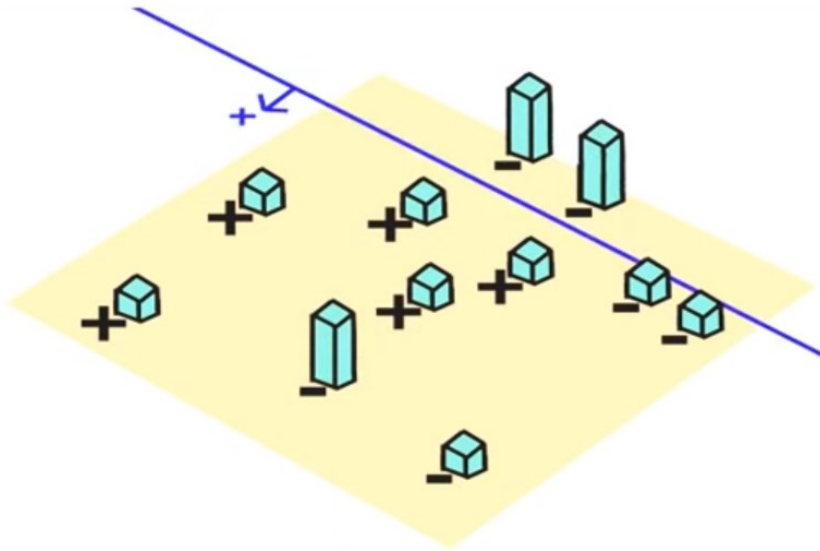
- Analiza $\rightarrow$ ima 3 *false positive* detekcije

AdaBoost algoritam

- Povećavamo težine (na slici visine) tim pogrešno detektiranim slikama s h_1
 - Zato što želimo naći sljedeći klasifikator/značajku koji se bolje ponaša za te slike koje su bile krivo klasificirane
 - Automatski smanjujemo težine slikama koje su ispravno klasificirane sa h_1



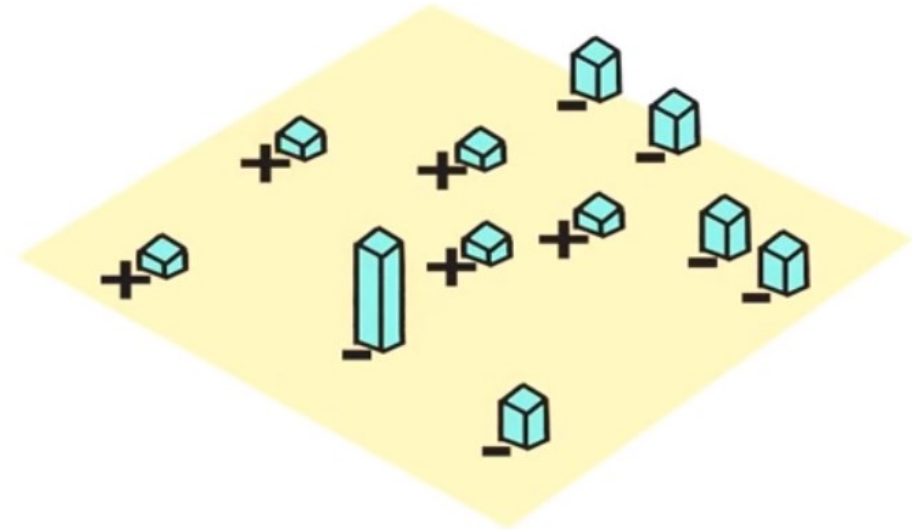
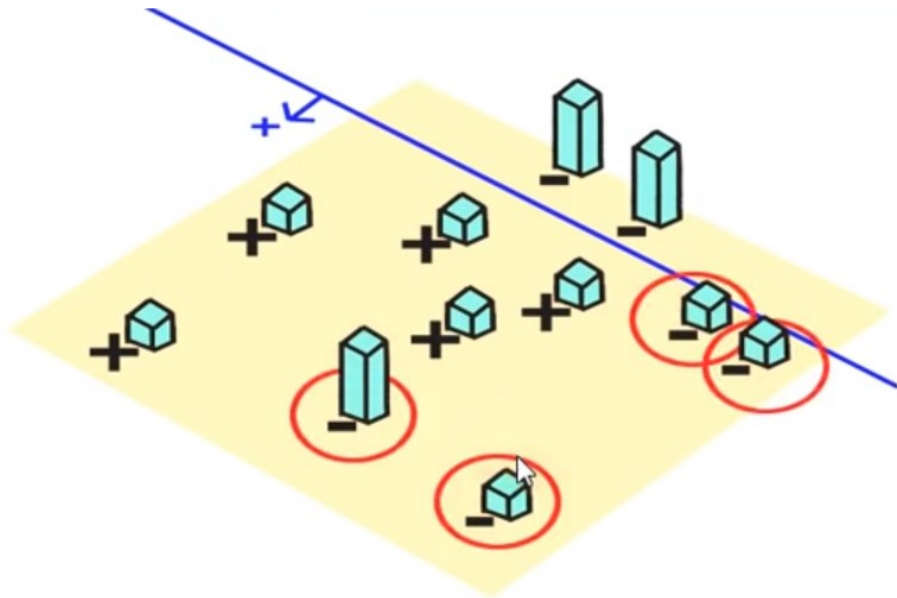
AdaBoost algoritam



- Klasifikator h_2 bolje klasificirao one 3 slike (2 od 3 je točno klasificirao)
- Ipak, on je ukupno 4 slike pogrešno klasificirao

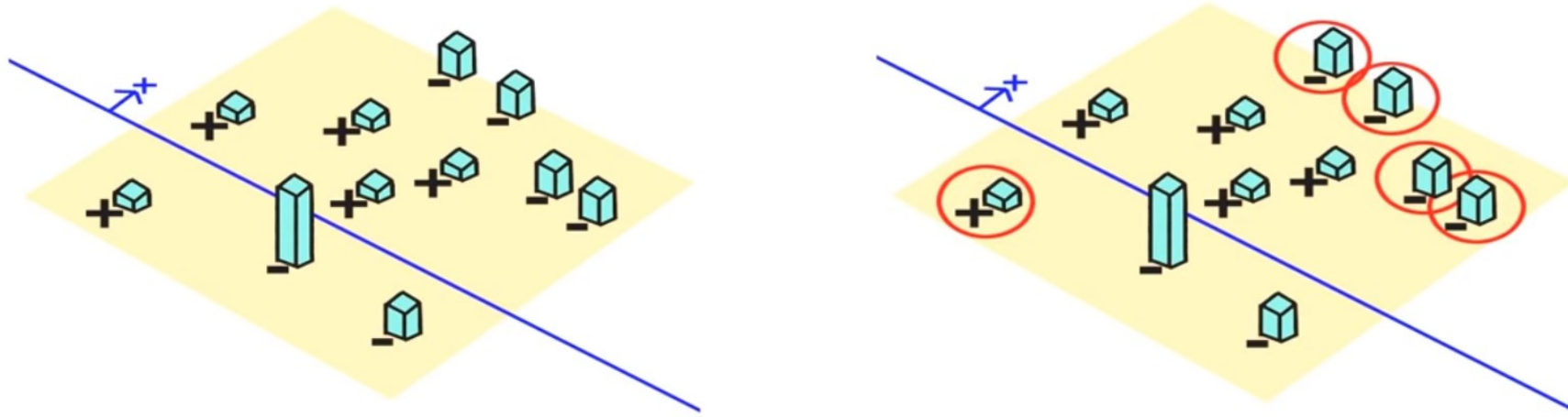
AdaBoost algoritam

- Povećavamo težine tim pogrešno detektiranim slikama s h_2
- Automatski smanjujemo težine slikama koje su ispravno klasificirane sa h_2



AdaBoost algoritam

- Tražimo novi klasifikator/značajku h_3 koja će sad te 4 pogrešno detektirane slike bolje klasificirati

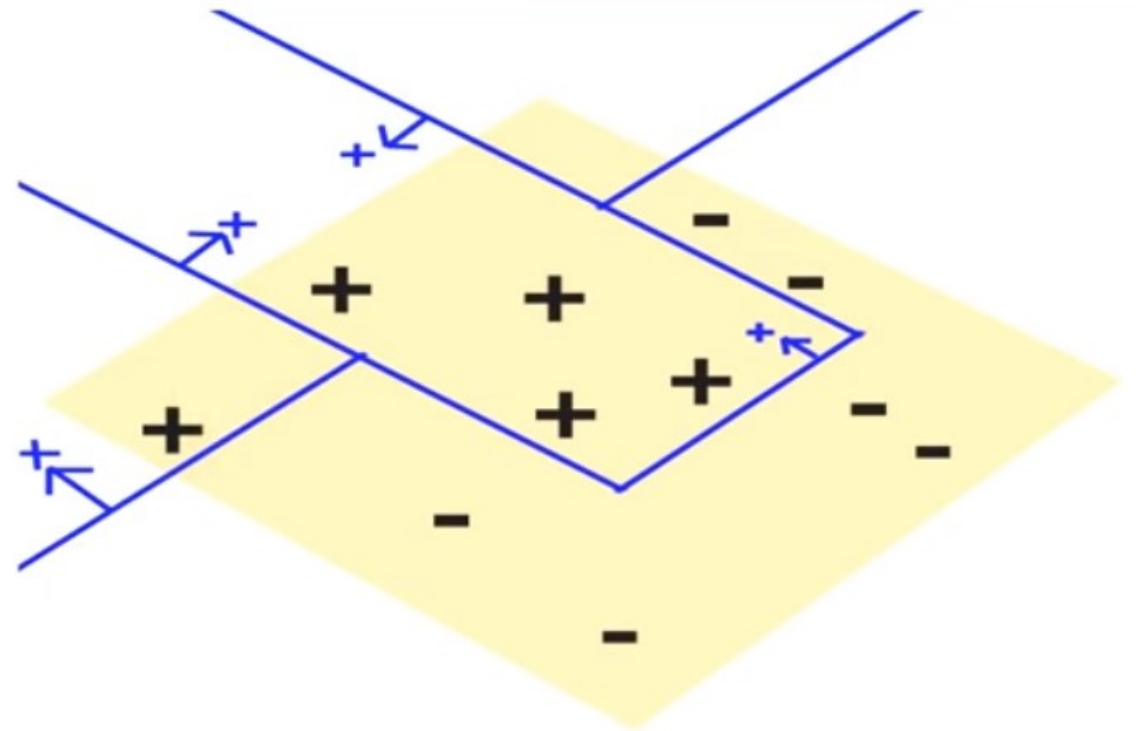


- Našli smo ga, ali on sad ima 5 pogrešnih odluka ...
- Ponavljamo postupak s povećavanjem i smanjivanjem težina ...

AdaBoost algoritam

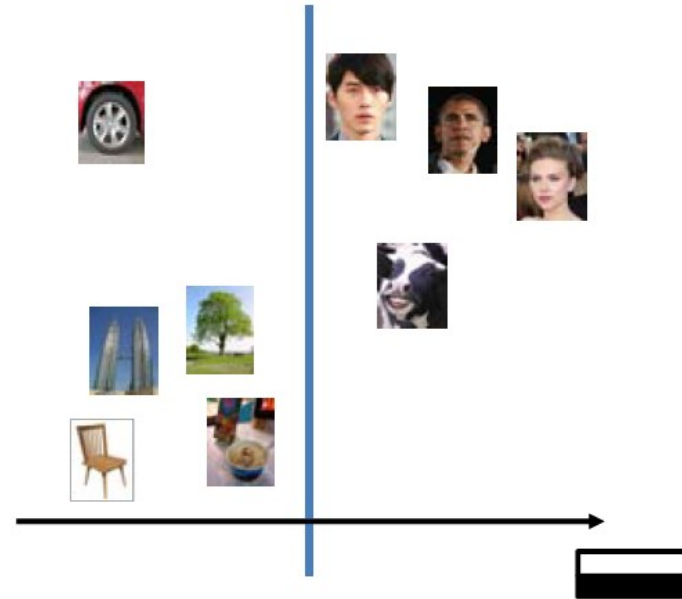
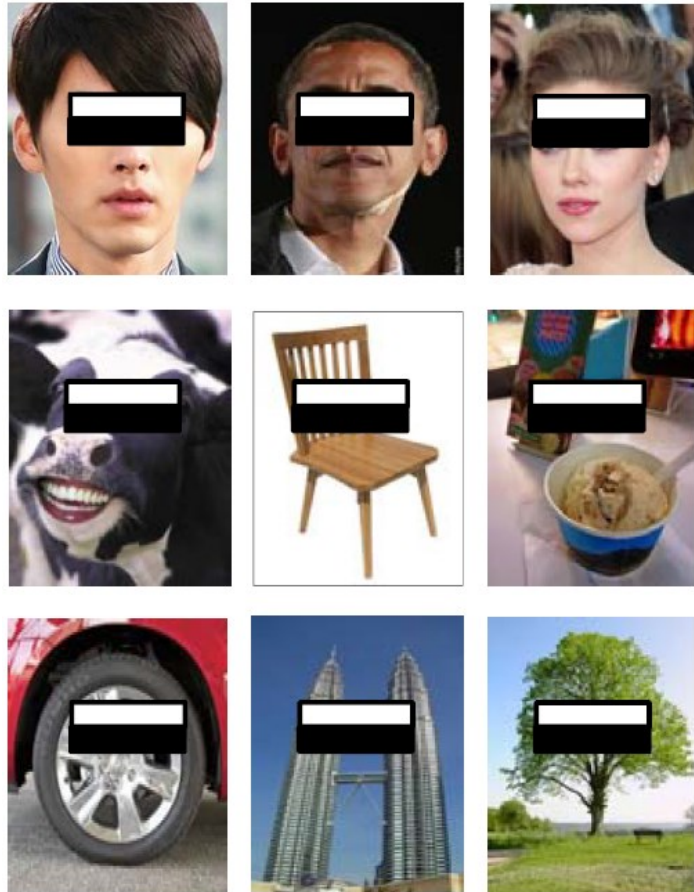
- Konačno, nakon zadanog broja iteracija radi se linearna kombinacija slabih klasifikatora koja uspješno detektira objekte u slici

$$H(x) = \sum_{n=1}^N a_n h_n(x)$$



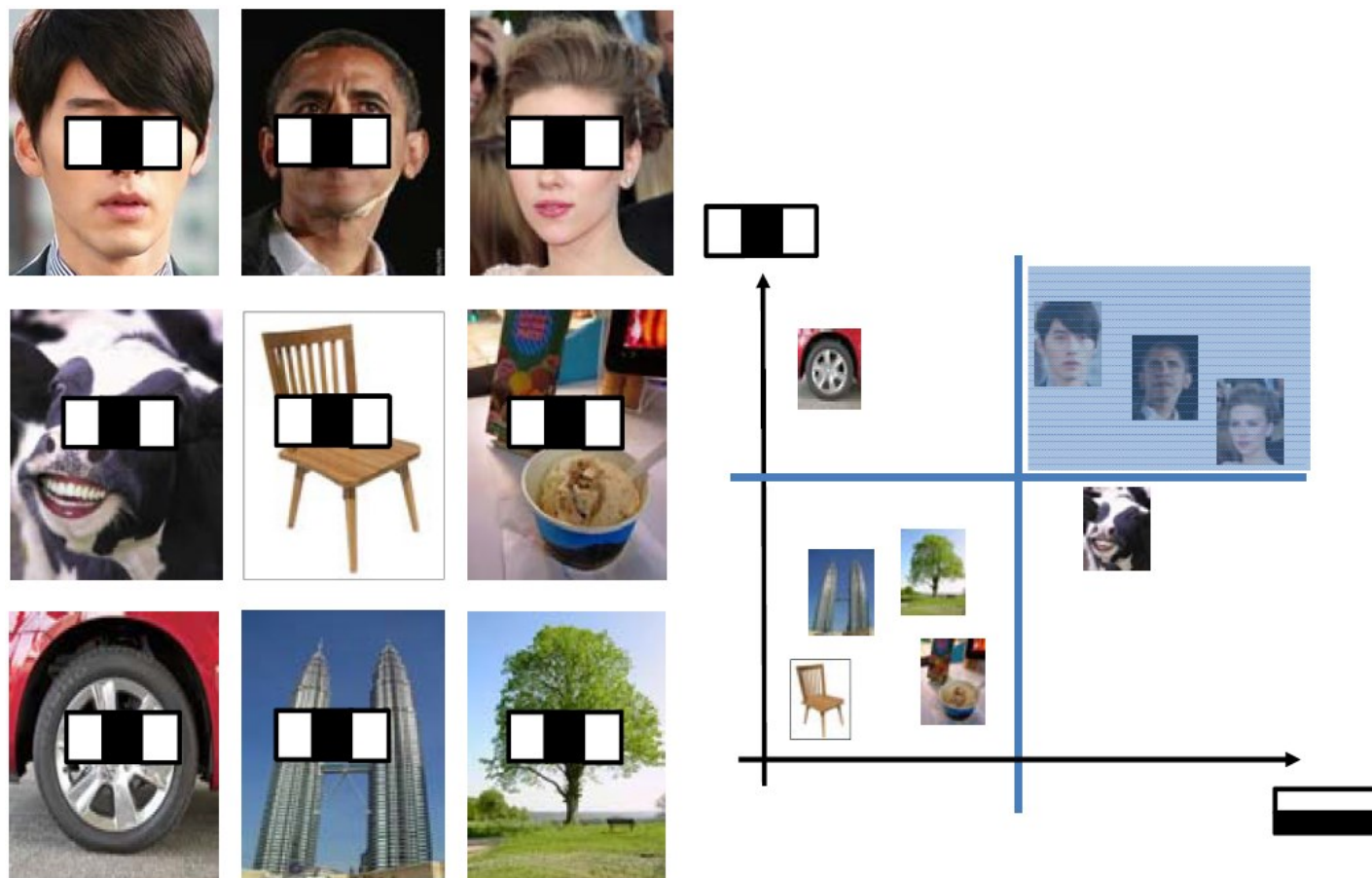
AdaBoost algoritam

- Jednostavan primjer sa samo dvije značajke (oči i nos)



AdaBoost algoritam

- Jednostavan primjer sa samo dvije značajke (oči i nos)



AdaBoost algoritam...zaključak

- Nijedan slabi klasifikator ne može samostalno donijeti ispravnu odluku za sve uzorke, ali linearna kombinacija slabih klasifikatora dovodi do puno boljih rezultata
- **Performanse inicijalnog Viola-Jones algoritma**
 - Već sa 200 slabih klasifikatora (značajki) detektira 95 % lica
 - 1 *false positive* na 14084 uzoraka
 - 2001. godine je skenirao sve podprozore 24x24 u slici dimenzija 384x288 elemenata slike za 0.7 sekundi s procesorom Pentium III na 700 MHz
 - **Nedovoljno za rad u stvarnom vremenu → Kako sad to? Zašto?**
- Detaljan način rada Adaboost algoritma i kompletna matematika samog postupka (ažuriranja težina, odabir značajki, ...) → [3], [4], [5]

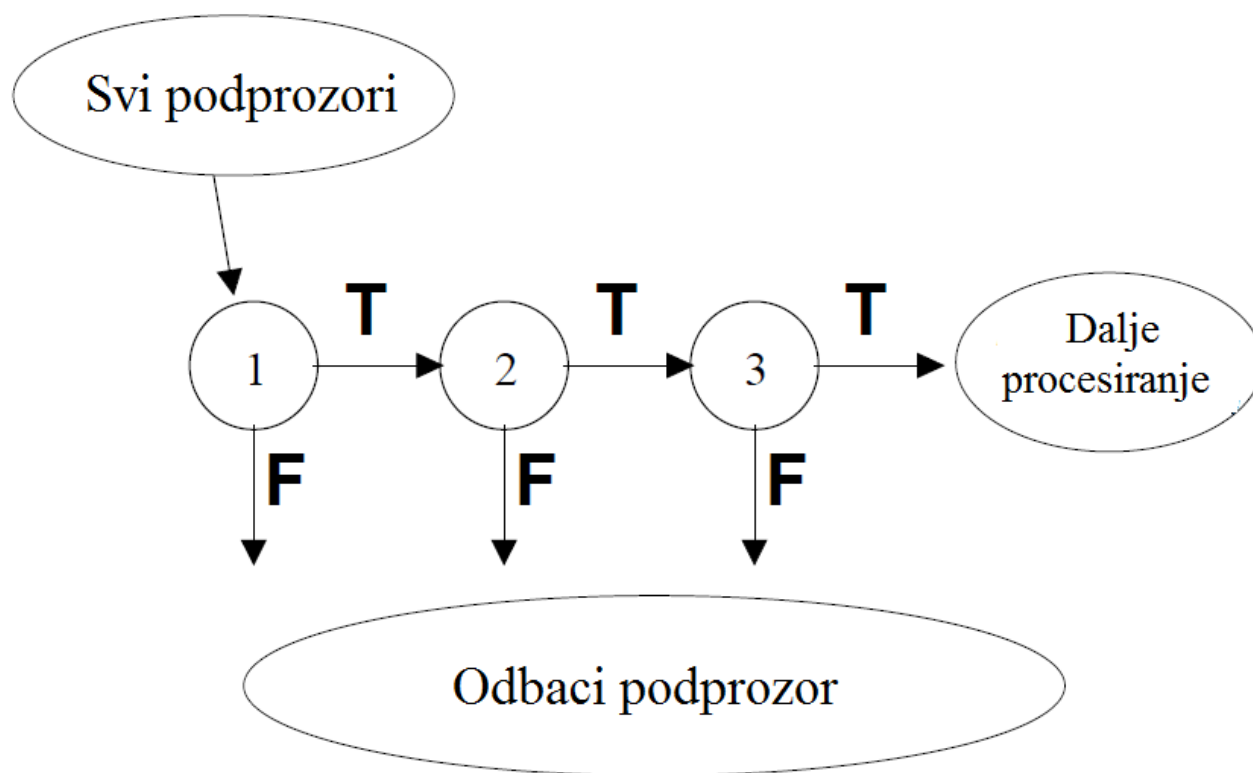
AdaBoost algoritam...zaključak

- **Rezultati eksperimenata pokazali** → jaki (boostani) klasifikator koji bi radio uspješno morao bi se sastojati od jako velikog broja značajki
- Obrada tako velikog broja značajki za svaku poziciju podprozora 24x24 → **neprikladno za rad u stvarnom vremenu**
- **Činjenica** → na većini lokacija unutar slike nema traženih objekata
- **Rješenje?**
 - Korištenje kaskade jakih klasifikatora → Ideja korištenja kaskade klasifikatora je koristiti više jakih klasifikatora koji će imati visoki postotak eliminacije lokacija u kojima se ne nalaze traženi objekti
 - Ulančavanjem takvih jakih klasifikatora u kaskadu znatno se ubrzava postupak detekcije

P i t a n j a ?

Kaskada klasifikatora

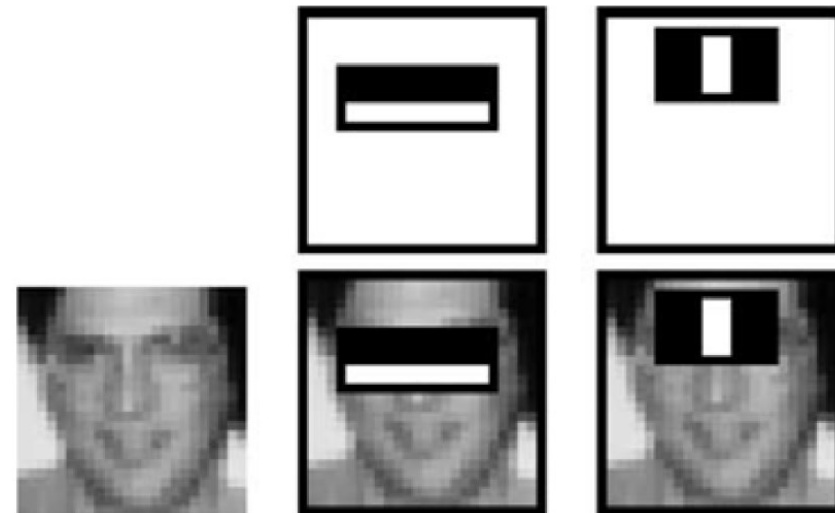
- Shema kaskade klasifikatora



- Prvi klasifikator eliminira velik broj negativnih uzoraka uz vrlo malo procesiranja (fokusira se pažnja na dijelove slike koji „obećavaju“)
- Sljedeće razine eliminiraju dodatne negativne uzorke, ali uz sve više procesiranja
- Nakon nekoliko razina broj podprozora za daljnju obradu je znatno smanjen

Kaskada klasifikatora

- Prvi jaki klasifikator (kod detekcije lica)
 - koristi samo 2 značajke
- Može biti podešen na razne načine, npr.
 - Detektira 100% lica
 - Da ima 50 % false positive rezultata



- Dalje će se procesirati samo *true positive* i *false positive* podprozori
- Ubrzanje rada → cilj je da se kroz cijelu kaskadu prođe na što manje lokacija
- Dublja kaskada (više jakih klasifikatora) → veća točnost detekcije

Kaskada klasifikatora

- OK, kaskada radi brže, ali kako odrediti parametre kaskade ???
 - Broj jakih klasifikatora (broj razina kaskade)
 - Broj slabih klasifikatora (značajki) u svakoj razini (svakom jakom klasifikatoru)
 - Prag za svaku razinu→ Pronalazak parametara (optimalne kombinacije) je složen proces
- Korisnik definira vrijednosti
 - f – maksimalno prihvatljiv *false positive rate* po razini
 - d – minimalno prihvatljiv *detection rate* po razini
 - F_{target} – ciljani ukupni *false positive rate*
 - P – skup pozitivnih primjera
 - N – skup negativnih primjera
 - $F_0=1.0$; $D_0=1.0$; $i = 0$

Kaskada klasifikatora

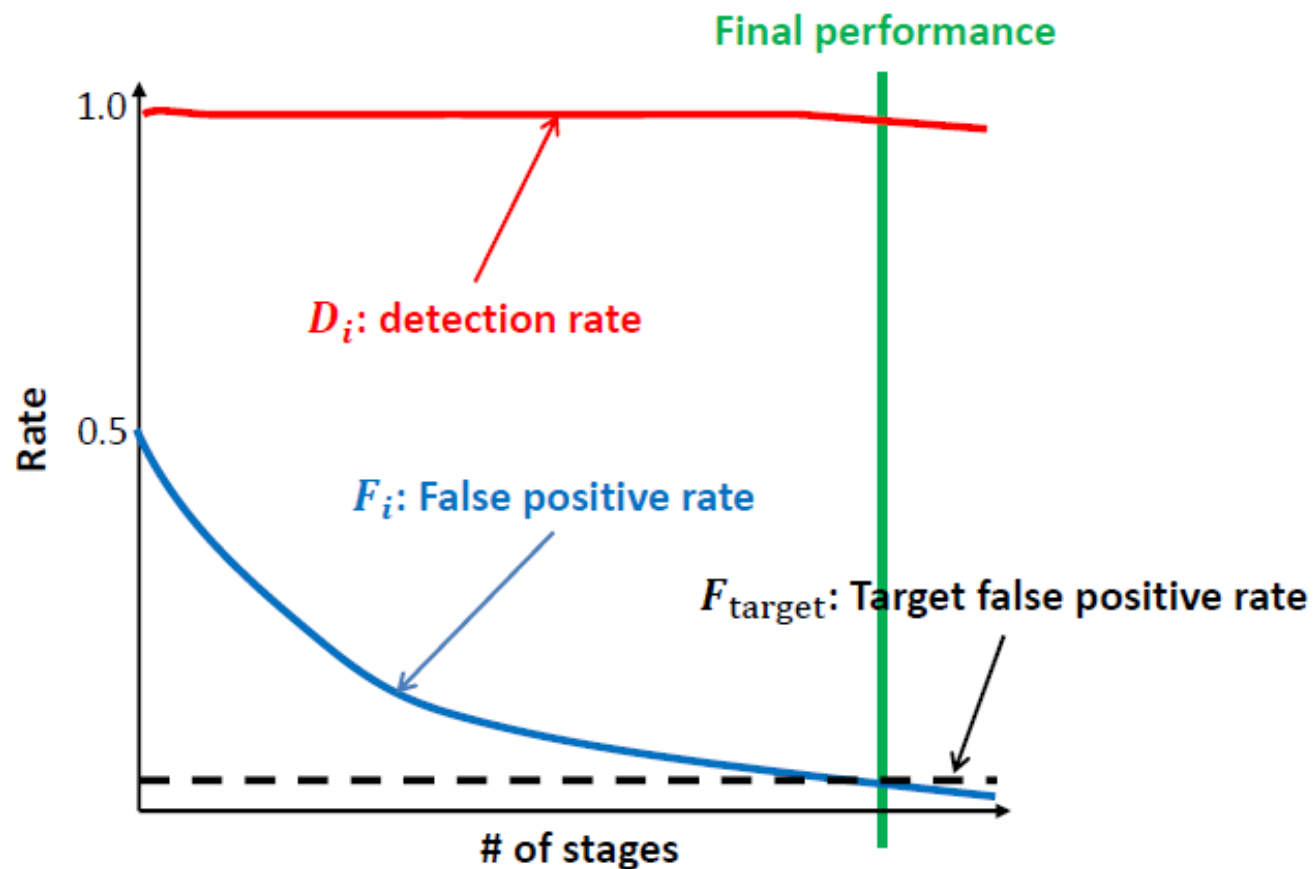
- Treniranje kaskadnih klasifikatora → detalji u predloženoj literaturi

- while $F_i > F_{target}$ **If FP rate is higher than the target FP rate, add a layer.**
 - $i \leftarrow i + 1$
 - $n_i = 0; F_i = F_{i-1}$
 - while $F_i > f \times F_{i-1}$ **Reduce false positive rate.**
 - * $n_i \leftarrow n_i + 1$ **Add weak classifiers in the current stage.**
 - * Use P and N to train a classifier with n_i features using **AdaBoost** **Use AdaBoost to find weak classifier.**
 - * Evaluate current cascaded classifier on validation set to determine F_i and D_i .
 - * Decrease threshold for the i th classifier until the current cascaded classifier has a detection rate of at least $d \times D_{i-1}$ (this also affects F_i) **Maintain good detection rate.**
 - $N \leftarrow \emptyset$
 - If $F_i > F_{target}$ then evaluate the current cascaded detector on the set of non-face images and put any false detections into the set N **Store hard negative examples.**

Training a cascade

Kaskada klasifikatora

- OK, kaskada radi brže, ali kako odrediti parametre kaskade ???

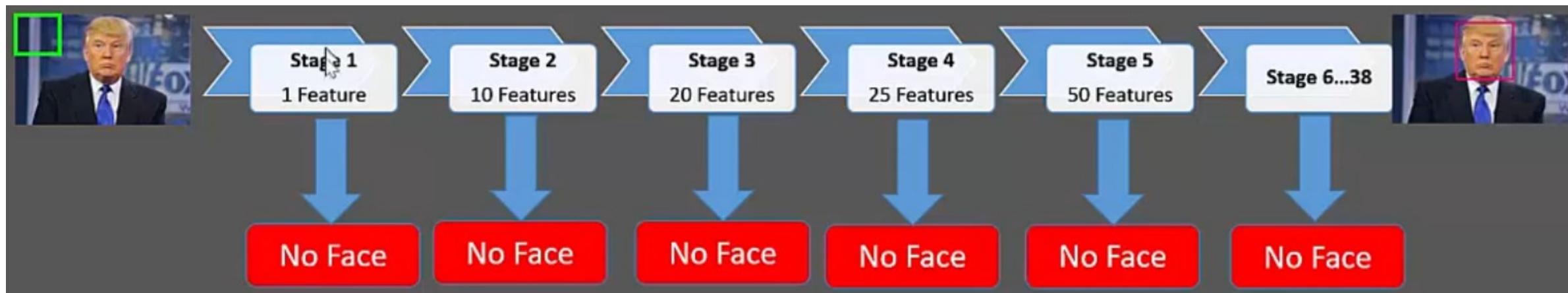


Izvorni Viola-Jones algoritam

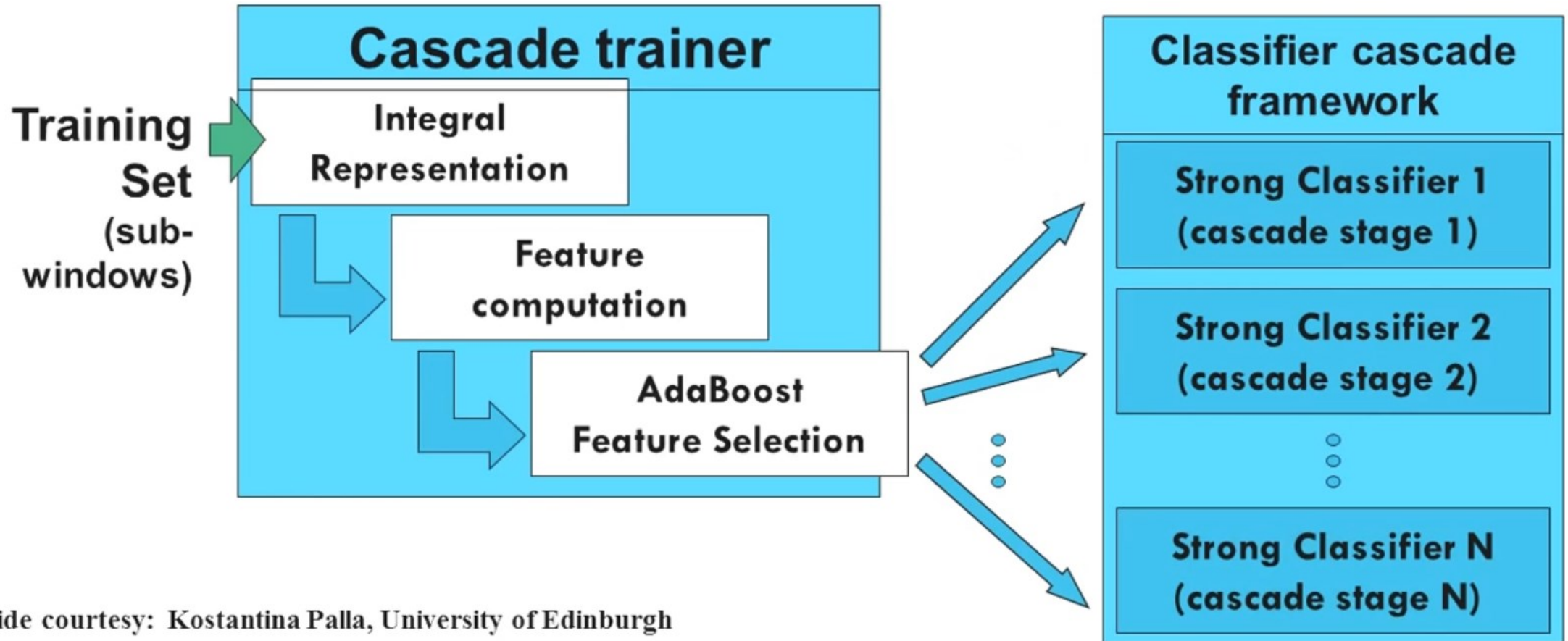
- 38 razina, ukupno 6060 značajki
- 1. razina
 - 2 značajke (najbolje prema Adaboost algoritmu)
 - 50 % *false positive rate* (f), 99.9 % *true positive rate* (d)
- 2. razina
 - 10 značajki
 - 20 % *false positive rate* (f), 99.9 % *true positive rate* (d)
- Sljedeće 2 razine → svaka 25 značajki
- Sljedeće 3 razine → svaka 50 značajki

Izvorni Viola-Jones algoritam

- 38 razina, ukupno 6060 značajki

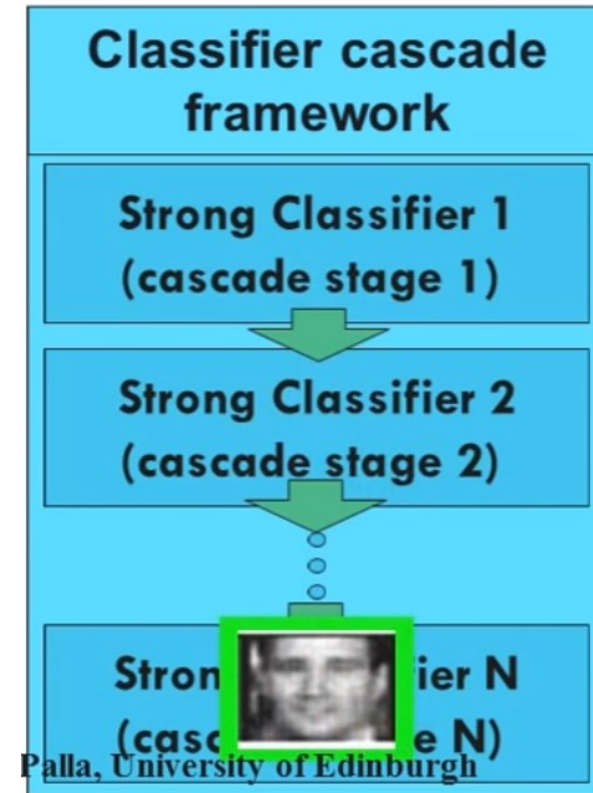
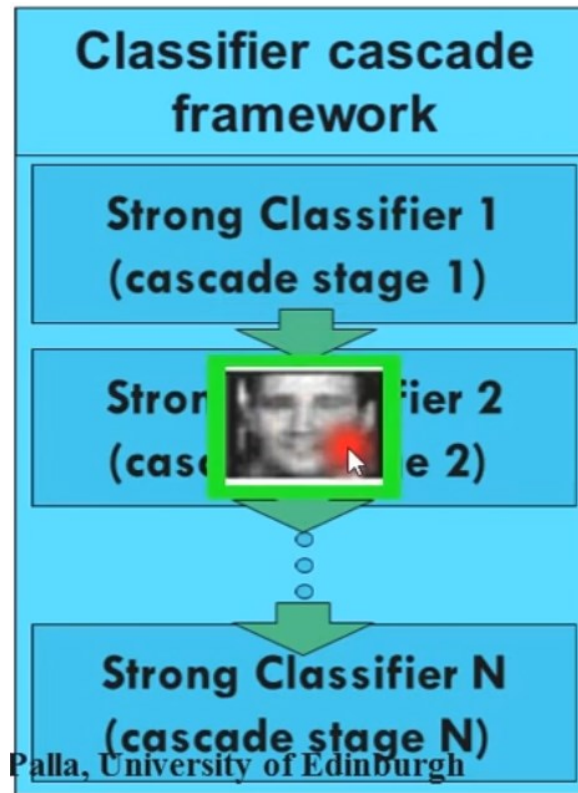
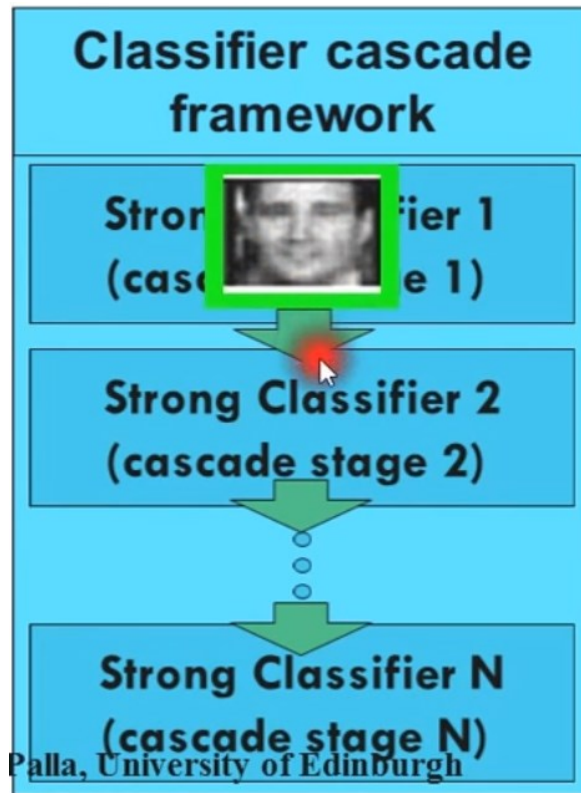
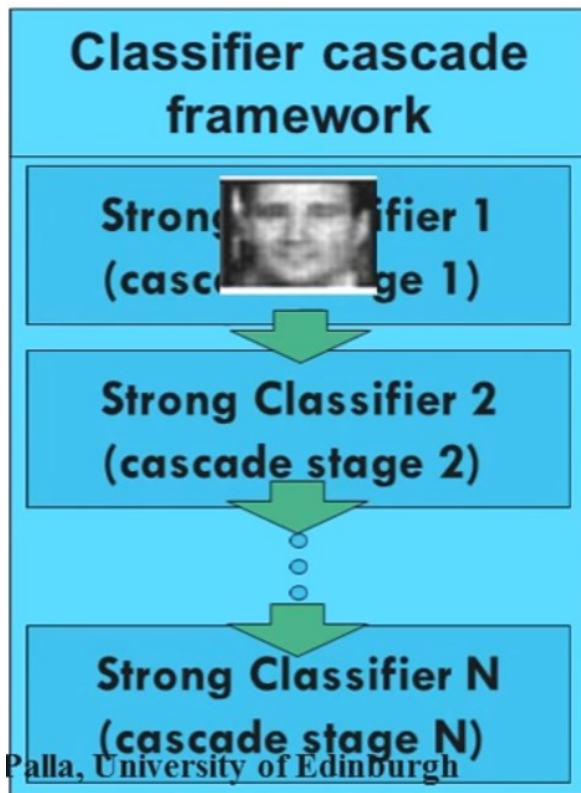


Da ponovimo ukratko fazu TRENIRANJA...



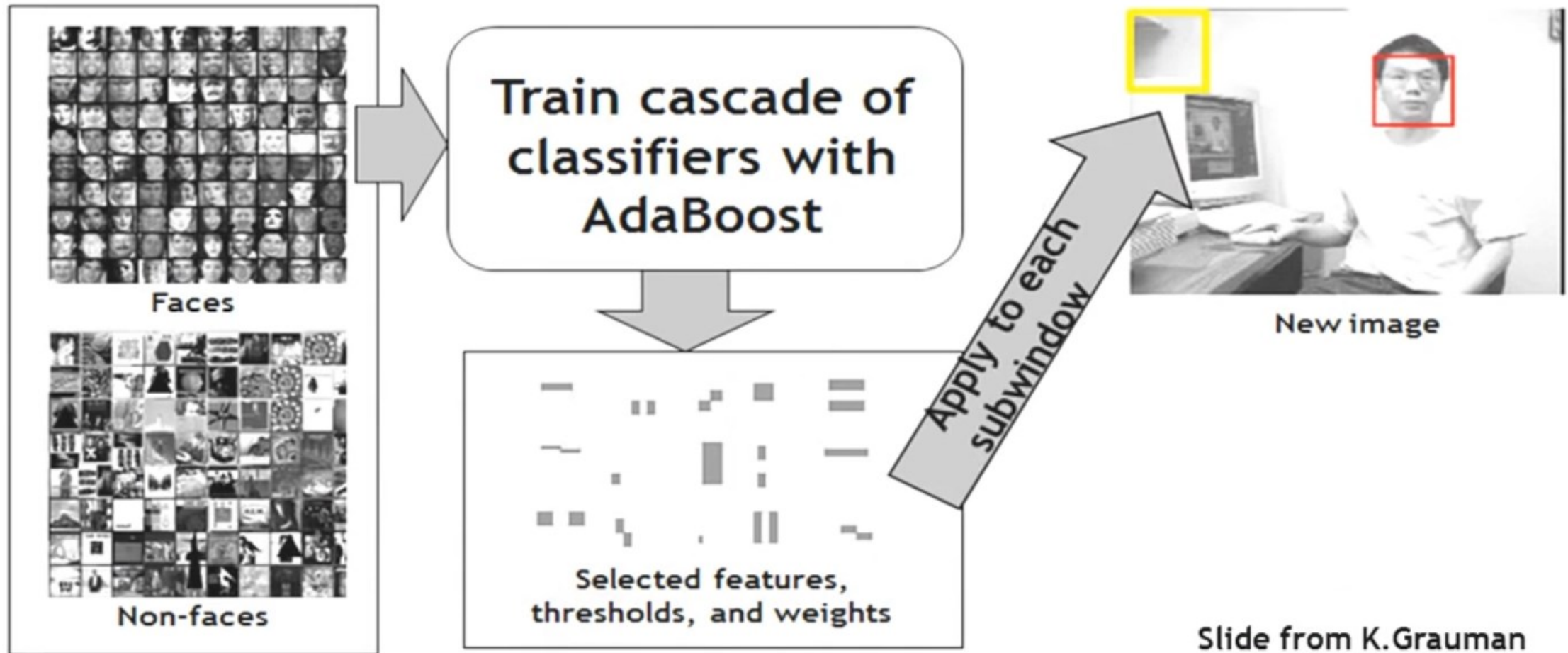
Slide courtesy: Kostantina Palla, University of Edinburgh

Faza TESTIRANJA..



FACE IDENTIFIED

Viola-Jones algoritam - Sažetak



Slide from K. Grauman

Konačno o Viola-Jones algoritmu za detekciju lica

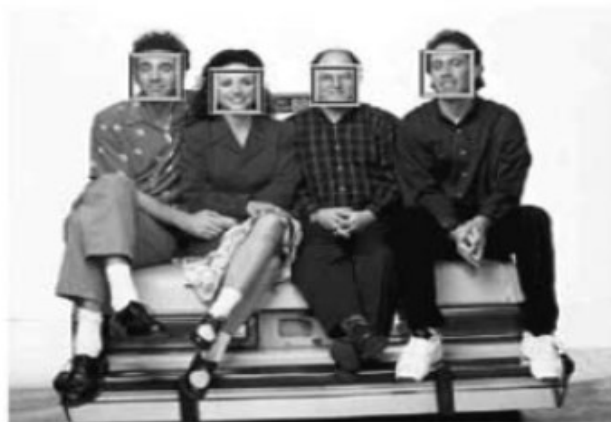
- **Vrline**

- Ekstremno brz proračun značajki
- Učinkovit odabir značajki
- Detektor invarijantan na skalu i lokaciju

- **Mane**

- Samo za slike lica sprijeda
- Ne može izaći na kraj s rotacijom
- Osjetljiv na uvjete osvjetljenja

Razultati...Primjeri...



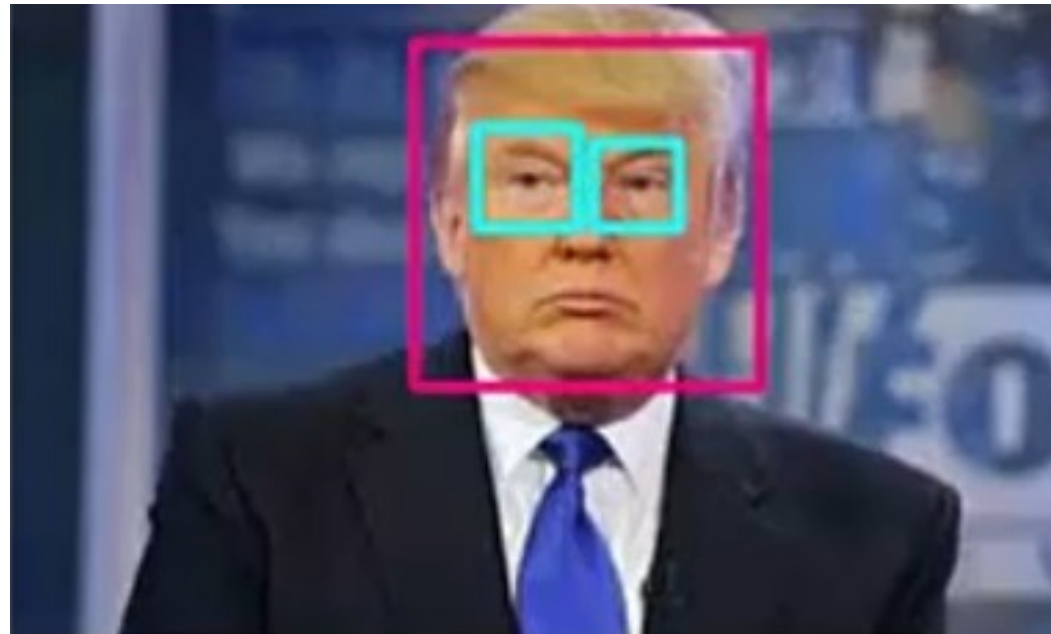
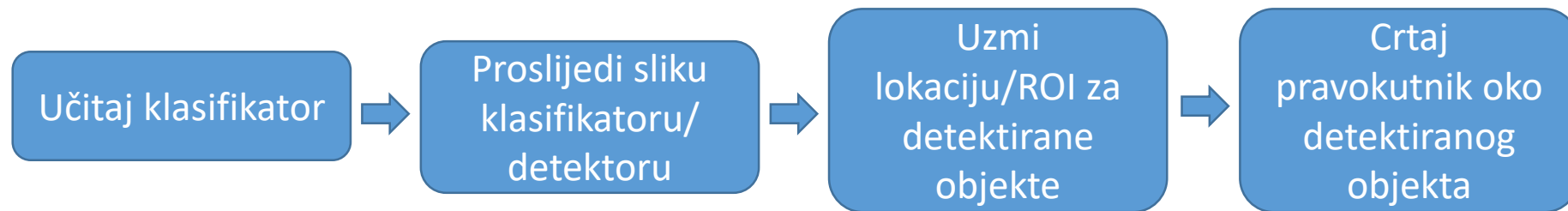
P i t a n j a ?

Detekcija lica i očiju pomoću Haarove kaskade (OpenCV)

Detekcija lica i očiju pomoću Haarove kaskade

- Koristimo neke prethodno istrenirane klasifikatore koji su dostupni u OpenCV
 - Spremljeni kao .XML datoteke
 - haarcascade_eye.xml
 - haarcascade_frontalface_default.xml
- Mogu se pronaći na
 - <https://github.com/opencv/opencv/tree/master/data/haarcascades>

Detekcija lica i očiju pomoću Haarove kaskade



Detekcija lica i očiju pomoću Haarove kaskade

- Primjer 1 (Python & OpenCV) – detekcija lica

```
import numpy as np
import cv2

# We point OpenCV's CascadeClassifier function to where our
# classifier (XML file format) is stored
face_classifier = cv2.CascadeClassifier('Haarcascades/haarcascade_frontalface_default.xml')

# Load our image then convert it to grayscale
image = cv2.imread('images/Trump.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Our classifier returns the ROI of the detected face as a tuple
# It stores the top left coordinate and the bottom right coordinates
faces = face_classifier.detectMultiScale(gray, 1.3, 5)

# When no faces detected, face_classifier returns an empty tuple
if faces is ():
    print("No faces found")

# We iterate through our faces array and draw a rectangle
# over each face in faces
for (x,y,w,h) in faces:
    cv2.rectangle(image, (x,y), (x+w,y+h), (127,0,255), 2)
    cv2.imshow('Face Detection', image)
    cv2.waitKey(0)

cv2.destroyAllWindows()
```

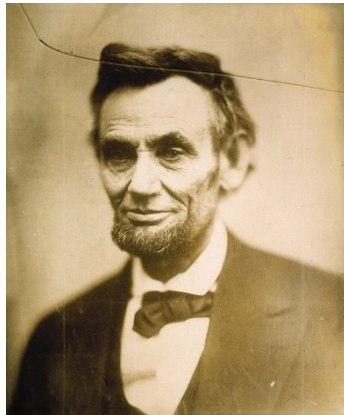
- Funkcija `cv2.CascadeClassifier` koristi prethodno istrenirani klasifikator → stvara se objekt *face_classifier*
- Koristimo metodu *detectMultiScale*
- To nam daje ROI za svako detektirano lice u slici
- *faces* je polje u kojem su lokacije za više lica pronađenih u slici (list of lists)
- Prolazimo kroz *faces* i crtamo pravokutnik oko svakog lica pronađenog u slici
- Probajmo nekoliko primjera...

Detekcija lica i očiju pomoću Haarove kaskade

- Primjer 1 (Python & OpenCV) – detekcija lica



Trump



abraham



bottlecaps



chihuahua



faceswap



obama



Sunflowers



zakom

Detekcija lica i očiju pomoću Haarove kaskade

- Primjer 2 (Python & OpenCV) – detekcija lica i očiju

```
import numpy as np
import cv2

face_classifier = cv2.CascadeClassifier('Haarcascades/haarcascade_frontalface_default.xml')
eye_classifier = cv2.CascadeClassifier('Haarcascades/haarcascade_eye.xml')

img = cv2.imread('images/Trump.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

faces = face_classifier.detectMultiScale(gray, 1.3, 5)

# When no faces detected, face_classifier returns an empty tuple
if faces is ():
    print("No Face Found")

for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(127,0,255),2)
    cv2.imshow('img',img)
    cv2.waitKey(0)
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = img[y:y+h, x:x+w]
    eyes = eye_classifier.detectMultiScale(roi_color)
    for (ex,ey,ew,eh) in eyes:
        cv2.rectangle(roi_color,(ex,ey),(ex+ew,ey+eh),(255,255,0),2)
        cv2.imshow('img',img)
        cv2.waitKey(0)

cv2.destroyAllWindows()
```

- Inicijaliziramo 2 klasifikatora
- Ako nema lica, oči se ne traže
- Ako ima lica, samo „izrezanu” sliku lica proslijeđujemo klasifikatoru očiju

Detekcija lica i očiju pomoću Haarove kaskade

- Primjer 3 (Python & OpenCV) – detekcija lica i očiju s web-kamere
- Kreiramo funkciju *face_detector* koja uzima sliku s web-kamere
 - Pronalazi lice u njoj
 - Izrezuje lice iz slike i samo to prikazuje
 - Pronalazi oči u izrezanoj slici
- Ne koristimo *waitKey* – želimo istovremeno imati lice i oči

```
import cv2
import numpy as np

face_classifier = cv2.CascadeClassifier('Haarcascades/haarcascade_frontalface_default.xml')
eye_classifier = cv2.CascadeClassifier('Haarcascades/haarcascade_eye.xml')

def face_detector(img, size=0.5):
    # Convert image to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_classifier.detectMultiScale(gray, 1.3, 5)
    if faces is ():
        return img

    for (x,y,w,h) in faces:
        x = x - 50
        w = w + 50
        y = y - 50
        h = h + 50
        cv2.rectangle(img, (x,y), (x+w,y+h), (255,0,0), 2)
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = img[y:y+h, x:x+w]
        eyes = eye_classifier.detectMultiScale(roi_color)

        for (ex,ey,ew,eh) in eyes:
            cv2.rectangle(roi_color, (ex,ey), (ex+ew,ey+eh), (0,0,255), 2)

    roi_color = cv2.flip(roi_color, 1)
    return roi_color

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    cv2.imshow('Our Face Extractor', face_detector(frame))
    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

cap.release()
cv2.destroyAllWindows()
```

Detekcija pješaka pomoću Haarove kaskade (OpenCV)

Detekcija pješaka pomoću Haarove kaskade

- Koristimo prethodno istrenirani klasifikator koji je dostupan u OpenCV
- Spremljen kao .XML datoteka
 - haarcascade_fullbody.xml

https://github.com/opencv/opencv/blob/master/data/haarcascades/haarcascade_fullbody.xml

Detekcija pješaka pomoću Haarove kaskade

- Primjer 4 (Python & OpenCV) – detekcija pješaka

```
import cv2
import numpy as np

# Create our body classifier
body_classifier = cv2.CascadeClassifier('Haarcascades\haarcascade_fullbody.xml')

# Initiate video capture for video file
cap = cv2.VideoCapture('images/walking.avi')

# Loop once video is successfully loaded
while cap.isOpened():

    # Read first frame
    ret, frame = cap.read()
    frame = cv2.resize(frame, None, fx=0.5, fy=0.5, interpolation = cv2.INTER_LINEAR)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # Pass frame to our body classifier
    bodies = body_classifier.detectMultiScale(gray, 1.2, 3)

    # Extract bounding boxes for any bodies identified
    for (x,y,w,h) in bodies:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 255), 2)
        cv2.imshow('Pedestrians', frame)

    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

cap.release()
cv2.destroyAllWindows()
```

- Slično kao što radi primjer za detekciju lica
- Nakon učitavanja okvira smanjujemo mu (*resize*) rezoluciju dvostruko kako bi ubrzali klasifikaciju
- Pritom koristimo jednostavnu interpolaciju da bi brže radilo (linearnu)
- Prilično dobro radi (nekad promaši, ali radi korektno)

Detekcija automobila pomoću Haarove kaskade (OpenCV)

Detekcija automobila pomoću Haarove kaskade

- Koristimo prethodno istrenirani klasifikator koji je dostupan u OpenCV
- Spremljen kao .XML datoteka
 - haarcascade_car.xml

http://www.codeforge.com/read/241845/cars3.xml_html

Detekcija automobila pomoću Haarove kaskade

- Primjer 5 (Python & OpenCV) – detekcija automobila

```
import cv2
import time
import numpy as np

# Create our body classifier
car_classifier = cv2.CascadeClassifier('Haarcascades\haarcascade_car.xml')

# Initiate video capture for video file
cap = cv2.VideoCapture('images/cars.avi')

# Loop once video is successfully loaded
while cap.isOpened():

    time.sleep(.05)
    # Read first frame
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Pass frame to our car classifier
    cars = car_classifier.detectMultiScale(gray, 1.4, 2)

    # Extract bounding boxes for any bodies identified
    for (x,y,w,h) in cars:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 255), 2)
        cv2.imshow('Cars', frame)

    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

cap.release()
cv2.destroyAllWindows()
```

- Prilično dobro radi (promaši nekad aute koji su daleko)
 - Ne detektira bicikl i kamion
 - Igranje s parametrima → pokušati popraviti performanse
 - Naredbe
 - `import time`
 - `time.sleep`
- Za eksperimentiranje s prikazom (brzinom prikaza)
- Isprobajmo s njima i bez njih

Viola-Jones algoritam

Korisna literatura

- [1] P. Viola, M. Jones, „Rapid Object Detection using a Boosted Cascade of Simple Features”, Proceedings of the 2001 IEEE Conference on computer vision and pattern recognition, 2001
- [2] P. Viola, M. Jones, „Robust Real-Time Face Detection”, International Journal of Computer Vision, vol. 57, no. 2, 2004, str 137-154
- [3] R. Meir, G. Rätsch, „An introduction to boosting and leveraging”, In Advanced lectures on machine learning , Springer, Berlin, Heidelberg, 2003
- [4] R. Goutham Golla, „Real-time face detection and tracking”
- [5] B. Han, „Boosting and face detection”, in Introduction to Computer Vision, 2015
- [6] OpenCV: Face Detection using Haar Cascades
https://docs.opencv.org/trunk/d7/d8b/tutorial_py_face_detection.html

P i t a n j a ?